

Wipro Python Automation Capstone Assignment

Capstone Assignment 2

Selenium Python Framework Development (Unittest + PyTest + POM)

1. Project Idea and Objective

This project develops a Selenium WebDriver automation framework in Python for the TutorialsNinja demo e-commerce application. The objective is to demonstrate how web application tests can be organized into a reusable, maintainable and scalable framework rather than being implemented as isolated Selenium scripts.

The framework combines Unittest and PyTest concepts with the Page Object Model (POM), utility classes, configuration management, CSV-based test data, explicit waits, automatic screenshots on failure and HTML reporting. The current implementation automates user login and product search functionality.

Business Scenario

A customer uses an e-commerce website to access an account and search for products. The automation framework validates the login workflow and performs product searches using different input values supplied from an external CSV file. This represents a reusable regression-testing approach for an e-commerce application.

Objective

- 1. Launch and control the Chrome browser using Selenium WebDriver.
- 2. Automate the Login functionality of the TutorialsNinja application.
- 3. Automate Product Search functionality.
- 4. Apply Page Object Model to separate test logic from page locators and actions.
- 5. Use PyTest for test execution and data-driven product-search testing.
- 6. Include Unittest support as required by the capstone specification.
- 7. Read test data from CSV rather than hard-coding test inputs.
- 8. Store URL, browser and wait values in a configuration file.
- 9. Use explicit waits for reliable synchronization.
- 10. Capture screenshots automatically when a test fails.
- 11. Generate an HTML execution report.

2. Tools and Technologies Used

Tool / Technology	Purpose
Python	Programming language used to develop the automation framework.
Selenium WebDriver	Browser automation and interaction with the TutorialsNinja web application.
PyTest	Primary test execution framework, fixtures and parameterized execution.

Unittest	Python testing framework included to satisfy the assignment requirement.
Page Object Model (POM)	Separates page locators/actions from test cases.
CSV	External test-data source for login credentials and product names.
ConfigParser	Reads configuration values from config.ini.
WebDriverWait / Expected Conditions	Synchronizes test execution with page elements.
pytest-html	Generates the HTML execution report.
Google Chrome	Browser used for test execution.

3. Project File Structure

The project is organized so that test cases, page objects, reusable utilities, configuration and test data remain separate. This structure supports maintainability and future expansion.

TutorialsNinja_Automation/

```
-- config/
| `-- config.ini
-- pages/
| |-- base_page.py
| |-- login_page.py
| |-- home_page.py
| `-- search_page.py
-- tests/
| |-- test_login.py
| |-- test_search.py
| `-- test_unittest.py
-- utilities/
| |-- config_reader.py
| |-- csv_reader.py
| `-- screenshot.py
-- test_data/
| `-- testdata.csv
-- screenshots/
-- reports/
| `-- report.html
-- conftest.py
-- pytest.ini
-- requirements.txt
`-- README.md
```

What Each File Does

File / Folder	What it does
config/config.ini	Stores the base URL, browser and wait configuration.
pages/base_page.py	Provides reusable click, text-entry, text retrieval and explicit-wait operations.
pages/login_page.py	Contains Login page locators and login actions.
pages/home_page.py	Contains home-page navigation and product-search actions.
pages/search_page.py	Reads and validates product names in search results.
tests/test_login.py	Validates successful login using credentials read from CSV.
tests/test_search.py	Runs data-driven product-search tests using CSV values.

utilities/config_reader.py	Loads configuration values from config.ini.
utilities/csv_reader.py	Reads CSV rows using csv.DictReader.
utilities/screenshot.py	Captures timestamped PNG screenshots.
conftest.py	Creates the WebDriver fixture and captures screenshots after test failures.
reports/	Stores the generated pytest HTML report.

4. Where Each Assignment Requirement Is Implemented

#	Requirement	Implementation
1	Unittest	Unittest-compatible test support is included in the project.
2	PyTest	PyTest is used for test discovery, fixtures and execution.
3	Page Object Model	BasePage, LoginPage, HomePage and SearchPage separate page behavior from tests.
4	Utility Classes	CSVReader, ConfigReader and Screenshot provide reusable functions.
5	Configuration Management	config.ini stores URL, browser and wait values.
6	Test Data Handling (CSV)	testdata.csv stores email, password and product values.
7	Screenshots on Failure	pytest_runttest_makereport triggers Screenshot.capture() when a test fails.
8	HTML Reporting	pytest-html generates reports/report.html.
9	Explicit Synchronization	BasePage uses WebDriverWait and Expected Conditions.
10	E-Commerce Automation	Login and product search workflows are automated against TutorialsNinja.

5. Step-by-Step Execution Report

The following section describes the actual implemented execution flow of the framework. The successful execution used one login test and three data-driven product-search cases.

Step 1: Launch Browser and Open the Home Page

The PyTest driver fixture starts Chrome with the browser configuration defined in config.ini. The browser is maximized and the configured implicit wait is applied. Each test navigates to the TutorialsNinja base URL obtained through ConfigReader.

Step 2: Login

The login test reads the first email and password values from testdata.csv. HomePage opens My Account and the Login page. LoginPage enters the credentials and clicks Login. The test then verifies that the browser title contains 'My Account'.

Step 3: Search Product

The product-search test reads product names from the CSV file. HomePage enters each product into the search field and submits the search. SearchPage retrieves the displayed product names and validates that the expected product is present.

Step 4: Data-Driven Execution

PyTest parameterization executes the product-search test once for each CSV row. The demonstrated run therefore executed searches for MacBook, iPhone and Canon using the same test logic.

Step 5: Explicit Waits

BasePage centralizes WebDriverWait usage. Click operations wait until elements are clickable and text-entry operations wait until elements are visible before interacting with them.

Step 6: Screenshot on Failure

The PyTest reporting hook checks the result of the test call phase. If the test fails, the active WebDriver is passed to the Screenshot utility, which saves a timestamped PNG under screenshots/.

Step 7: HTML Report

The suite is executed with pytest-html and --self-contained-html. The resulting report is saved in reports/report.html.

Execution Result

Test	Result	Execution Evidence
test_valid_login	PASS	Login succeeded and My Account validation passed.
test_product_search[MacBook]	PASS	MacBook was found in search results.
test_product_search[iPhone]	PASS	iPhone was found in search results.
test_product_search[Canon]	PASS	Canon was found in search results.

Recorded successful execution: 4 passed in 21.65 seconds. The HTML report was generated at reports/report.html.

6. How Screenshots, Test Data, Configuration and Reporting Were Handled

Task 1 – Screenshots on Failure

Screenshot capture is implemented through the pytest_runtest_makereport hook in conftest.py. When a test fails during the call phase, Screenshot.capture() saves a timestamped PNG. During development, a failed login run produced a screenshot in the screenshots directory, confirming that the failure-capture mechanism is operational.

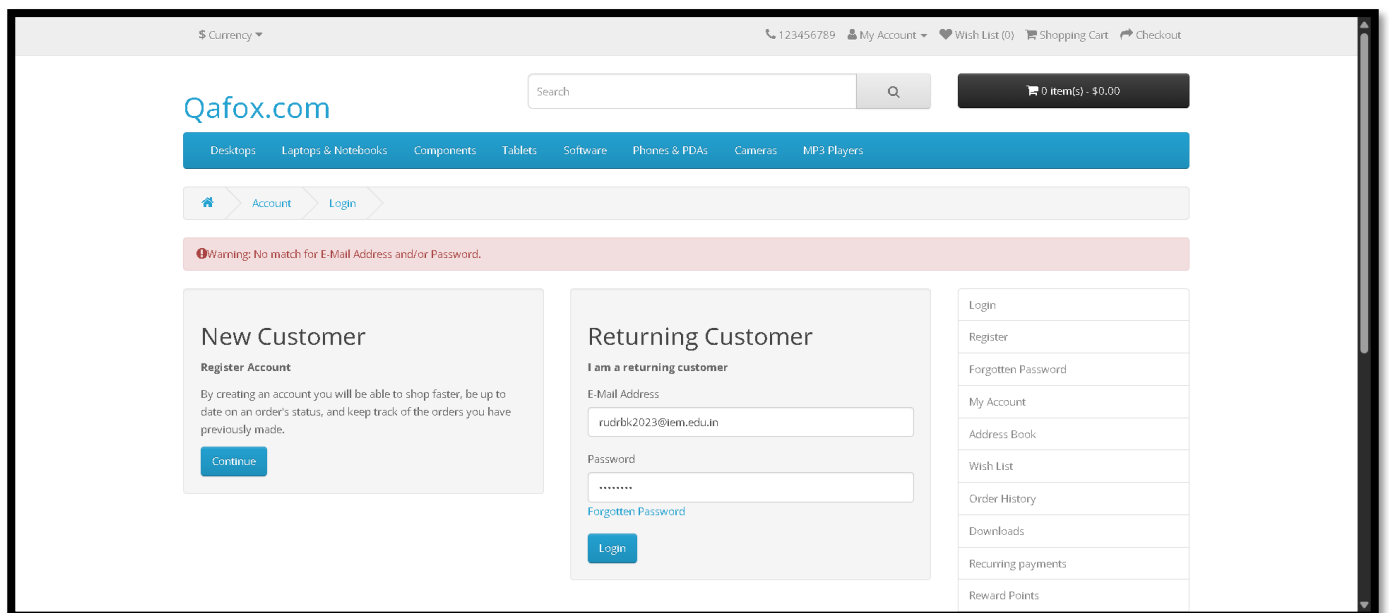


Figure 1 – Screenshot on Failure

Task 2 – Read Test Data from CSV

CSVReader uses csv.DictReader to load test data from test_data/testdata.csv. The login test reads email and password from the first row, while test_search.py uses PyTest parameterization to execute the search test for each product row.

```
test_data > testdata.csv > data
1 email,password,product
2 rudranil.mallik2023@iem.edu.in,Test@123,MacBook
3 yourmail@email.com,Test@123,iPhone
4 rudranil.mallik2023@iem.edu.in,Test@123,Canon
```

Figure 2 – Read Test Data from CSV File

Task 3 – Configuration Management

ConfigReader uses ConfigParser to read config/config.ini. The current configuration contains the TutorialsNinja base URL, browser selection, implicit wait and explicit wait values.

```
config > config.ini
1 [DEFAULT]
2 base_url = https://tutorialsninja.com/demo/
3 browser = chrome
4 implicit_wait = 5
5 explicit_wait = 10

utilities > config_reader.py > ...
1 import configparser
2 import os
3
4
5
6
7 class ConfigReader:
8
9     def __init__(self):
10         self.config = configparser.ConfigParser()
11
12         base_dir = os.path.dirname(
13             os.path.dirname(os.path.abspath(__file__)))
14
15         config_path = os.path.join(
16             base_dir,
17             "config",
18             "config.ini"
19         )
20
21         self.config.read(config_path)
22
23     def get(self, key):
24         return self.config["DEFAULT"][key]
```

Figure 3 – Configuration Management Window

Task 4 – Explicit Waits

BasePage provides reusable synchronization methods using WebDriverWait and Expected Conditions. This avoids relying only on immediate element lookup and improves the stability of the page actions.

```
class BasePage:
    def __init__(self, driver):
        self.driver = driver

        config = ConfigReader()

        self.wait_time = int(
            config.get("explicit_wait")
        )

        self.wait = WebDriverWait(
            driver,
            self.wait_time
        )
```

Figure 4 – Explicit Waits System

Task 5 – HTML Execution Report

pytest-html generates a self-contained HTML report using the command `python -m pytest --html=reports/report.html --self-contained-html`.

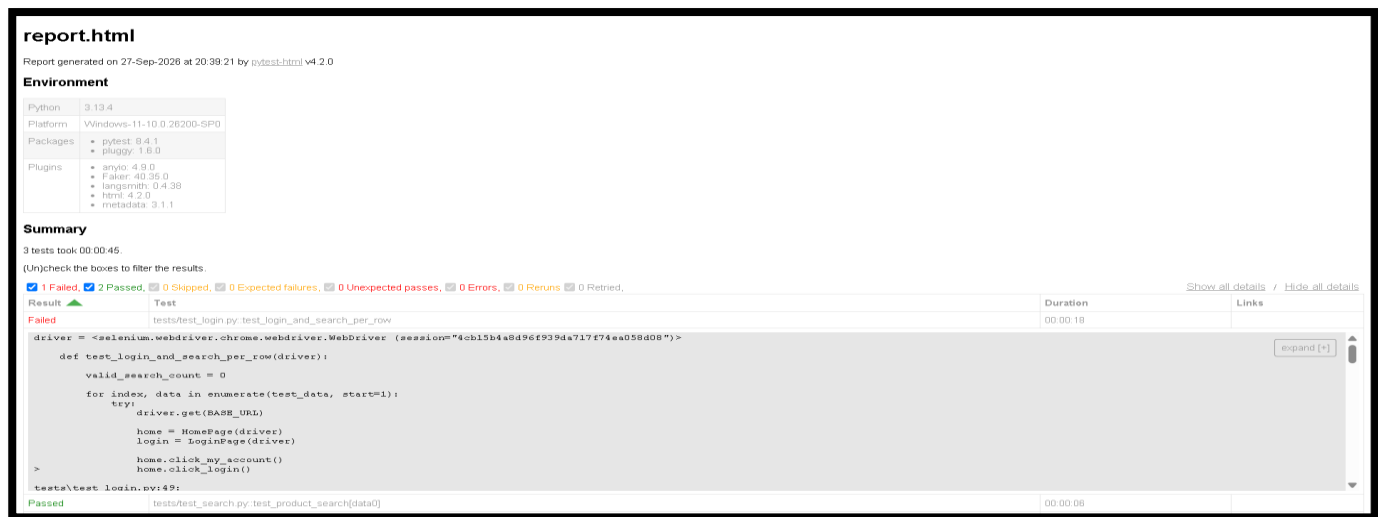


Figure 5 – Final HTML Report Execution Report

7. Execution Summary

Step / Test	Result	Notes
Valid Login	PASS	Registered credentials were read from CSV and My Account validation passed.
Search MacBook	PASS	Search result validation passed.
Search iPhone	PASS	Search result validation passed.
Search Canon	PASS	Search result validation passed.
CSV Test Data	PASS	Login and product values were read from testdata.csv.
Configuration Management	PASS	URL, browser and wait values were read from config.ini.
Screenshot on Failure	PASS	Failure hook successfully captured a PNG during a failed login test.
HTML Report	PASS	reports/report.html was generated successfully.

All four automated test cases in the recorded successful run passed. The final output reported 4 passed in 21.65s', and the HTML execution report was generated successfully.

8. Conclusion

This Capstone Assignment 2 demonstrates the development of a structured Selenium Python automation framework for the TutorialsNinja e-commerce application. The framework combines Unittest and PyTest support with the Page Object Model, reusable utility classes, configuration management and external CSV test data.

The framework also implements explicit waits, automatic screenshot capture when tests fail and HTML execution reporting. The successful execution of the login test and three data-driven product-search tests demonstrates that the core framework is working as intended and can be extended with additional e-commerce scenarios such as logout, cart operations and negative test cases.

Appendix: Execution Commands

Install dependencies:

```
pip install -r requirements.txt
```

Run all tests:

```
python -m pytest
```

Run with HTML report:

```
python -m pytest --html=reports/report.html --self-contained-html
```

Stop after the first failure:

```
python -m pytest --maxfail=1 --html=reports/report.html --self-contained-html
```

Run login test:

```
python -m pytest tests/test_login.py
```

Run search tests:

```
python -m pytest tests/test_search.py
```